ДИСЦИПЛИНА **Математическая логика и теория алгоритмов**
(полное наименование дисциплины без сокращений)

ИНСТИТУТ **информационных технологий**

КАФЕДРА **прикладной математики**
(полное наименование кафедры)

ВИД УЧЕБНОГО МАТЕРИАЛА **Лекция**
(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **Батенков Кирилл Александрович**
(фамилия, имя, отчество)

СЕМЕСТР **1, 2024-2025**
(указать семестр обучения, учебный год)

1. ОСНОВЫ АЛГОРИТМИЗАЦИИ, ЧАСТЬ 2

1.1. Методы повышения эффективности алгоритмов

Сегодня наиболее интересные результаты в разработке эффективных алгоритмов связаны с привлечением аппарата других научных дисциплин, в частности – биологии и теоретических результатов специальных разделов современной математики [26].

I. Алгоритмы и простые числа (Алгоритм Рабина – Карпа)

Одна из нетривиальных идей использования простых чисел для разработки и доказательства эффективности алгоритмов - идея предложенного в 1981 и опубликованного в 1987 году Рабином и Карпом алгоритма поиска подстроки в строке. Центральная идея алгоритма связана с переходом к сравнению чисел вместо посимвольного сравнения образца и части строки поиска.

Пусть дана символьная строка T длиной m (m-битовая строка) и образец P длиной n. Нам необходимо найти все вхождения образца в строку. Обозначим через T_r подстроку длиной n, начиная с бита r. Определим

$$H(P) = \sum_{i=1}^{n} 2^{n-i} \cdot P(i), \quad H(T_r) = \sum_{i=1}^{n} 2^{n-i} \cdot T(r+i-1).$$

Мы рассматриваем тем самым подстроки как двоичные числа, очевидно, что образец P входит в строку T, начиная с позиции r, если $H(P) = -H(T_r)$. Таким образом, мы сравниваем числа, а не символы, и если вычисление $H(T_r)$ при сдвиге может быть выполнено за $O(1)$, то сложность алгоритма составит $O(n+m)$, т.е. теоретическую границу сложности задачи поиска. Принципиальная проблема этой идеи экспоненциальный рост чисел $H(P)$ и $H(T_r)$, с ростом длины образца. При обычном байтовом кодировании символов образец всего из 12 букв занимает 96 бит, и, следовательно, $H(P) < 2^{96} - 1$. Необходимо было сохранить внешнюю привлекательность идеи, но уложиться в рамки реального диапазона представления целых чисел (машинного слова) в компьютере для образцов любой длины.

Решение проблемы, предложенное Рабином и Карпом, состоит в использовании остатков от деления $H(P)$ и $H(T_r)$ на некоторое простое число p. Эти остатки называются дактилограммами. Но действительная мощность предложенного метода состоит в доказательстве того, что вероятность ошибки (ложного срабатывания) может быть сделана малой, если простое число выбирается случайно из некоторого множества. Обозначим через

$$H_p(P) = T(P) \bmod p, \quad H_p(T_r) = H(T_r) \bmod p$$

Однако если вычислять вначале $H(P)$, а затем брать остаток, то мы снова окажемся за пределами машинного слова. Для эффективного вычисления $H_p(P)$ и $H_p(T_r)$ может быть использована схема Горнера, в результате чего во время

вычислений ни один из промежуточных результатов не будет больше $2 \cdot p$. Схема вычислений имеет вид

$$H_p(P) =$$
$$= \left(\left(\left(\left(P(1) \cdot 2 \bmod p + P(2)\right) \cdot 2 \bmod p + P(3)\right) \cdot 2 \bmod p\right) + \cdots + P(n)\right) \bmod p.$$

Следующая задача на пути создания эффективного алгоритма – быстрое вычисление сдвига значения $H(T_r)$. Заметим, что

$$H(T_r) = 2 \cdot H(T_{r-1}) \quad - T \cdot T(r-1) + T(r+n+1),$$

и, выполняя вычисления по $\bmod p$, мы можем вычислить $H_p(T_r)$ по $H_p(T_{r-1})$ с трудоемкостью $O(1)$. Очевидно, что $H_p(P)$ и $H_p(T_1)$ вычисляются однократно со сложностью $O(n)$. При каждом сдвиге по строке мы вычисляем $H_p(T_r)$ и сравниваем числа, что дает по порядку $O(m-n)$, и мы получаем общую линейную сложность. Обоснование того, что вероятность ложного совпадения (возможно, что числа $H(P)$ и $H(T_r)$ различны, в то время как $H_p(P)$ и $H_p(T_r)$ совпадают) мала, происходит на основе теорем и лемм теории простых чисел. Центральная теорема подхода Рабина – Карпа формулируется следующим образом.

Теорема. Пусть P и T некоторые строки, причем $n \cdot m > 29$, где $n = |P|$, $m = |T|$. Пусть I – некоторое положительное число. Если $p -$ случайно выбранное простое число, не превосходящее I, то вероятность ложного совпадения P и T не превосходит $\pi(n \cdot m)/\pi(I)$, где функция $\pi(n)$ в теории простых чисел обозначает количество простых чисел, не превосходящих n, например, $\pi(12) = 5$, т. к. существует 5 простых чисел не превосходящих 12. Если выбрать $I = n \cdot m^2$, то вероятность ложного совпадения не превосходит $2{,}53/m$.

Алгоритм Рабина-Карпа:

1. Выбрать положительное целое I (например, $I = n \cdot m^2$).

2. Случайно выбрать простое число p, не превосходящее I (например, используя тест Миллера (Рабина).

3. По схеме Горнера вычислить $H_p(P)$ и $H_p(T_1)$.

4. Для каждой позиции r в $T(1 < r < m - n + 1)$ вычислить $H_p(T_r)$ и сравнить с $H_p(P)$. Если они равны, то либо объявить о вероятном совпадении, либо выполнить явную проверку, начиная с позиции r.

Существует несколько модификаций метода Рабина – Карпа, связанных с обнаружением ошибок. Один из них основан на методе Бойера-Мура (см. 5.1.3. основного текста) и со сложностью $O(m)$ либо подтверждает отсутствие ложных совпадений, либо декларирует, что, по меньшей мере, одно из совпадений ложное. В случае ложных совпадений нужно выполнять алгоритм заново с новым значением p, вплоть до отсутствия ложных совпадений. Тем самым алгоритм Рабина – Карпа преобразуется из метода с малой вероятностью ошибки и со сложностью $O(m+n)$ в худшем случае в метод, который не делает ошибок, но обладает ожидаемой

сложностью $O(m+n)$ – это преобразование алгоритма Монте-Карло в алгоритм Лас-Вегаса.

Отметим, что метод Рабина – Карпа успешно применяется (наряду с целым рядом других алгоритмов поиска подстрок) в таком разделе современной науки как вычислительная молекулярная биология, для поиска совпадающих цепочек ДНК и расшифровки генов.

II. Генетические алгоритмы

Практическая необходимость решения ряда NP -полных задач в оптимизационной постановке при проектировании и исследовании сложных систем привела разработчиков алгоритмического обеспечения к использованию биологических механизмов поиска наилучших решений. В настоящее время эффективные алгоритмы разрабатываются в рамках научного направления, которое можно назвать «природные вычисления», объединяющего такие разделы, как генетические алгоритмы, эволюционное программирование, нейросетевые вычисления, клеточные автоматы и ДНК-вычисления, муравьиные алгоритмы. Исследователи обращаются к природным механизмам, которые миллионы лет обеспечивают адаптацию биоценозов к окружающей среде. Одним из таких механизмов, имеющих фундаментальный характер, является механизм наследственности. Его использование для решения задач оптимизации привело к появлению генетических алгоритмов.

В живой природе особи в биоценозе конкурируют друг с другом за различные ресурсы, такие, как пища или вода. Кроме того, особи одного вида в популяции конкурируют между собой, например, за привлечение брачного партнера. Те особи, которые более приспособлены к окружающим условиям, будут иметь больше шансов на создание потомства. Слабо приспособленные либо не произведут потомства, либо их потомство будет очень немногочисленным. Это означает, что гены от высоко приспособленных особей будут распространяться в последующих поколениях. Комбинация хороших характеристик от различных родителей иногда может приводить к появлению потомка, приспособленность которого даже больше, чем приспособленность его родителей. Таким образом, вид в целом развивается, лучше и лучше приспосабливаясь к среде обитания.

Алгоритм решения задач оптимизации, основанный на идеях наследственности в биологических популяциях, был впервые предложен Джоном Холландом (1975 г.). Он получил название репродуктивного плана Холланда, и широко использовался как базовый алгоритм в эволюционных вычислениях. Дальнейшее развитие эти идеи, как собственно и свое название - генетические алгоритмы, получили в работах Гольдберга и Де Йонга.

Цель генетического алгоритма при решении задачи оптимизации состоит в том, чтобы найти лучшее возможное, но не гарантированно оптимальное решение. Для реализации генетического алгоритма необходимо выбрать подходящую структуру

данных для представления решений. В постановке задачи поиска оптимума, экземпляр этой структуры должен содержать информацию о некоторой точке в пространстве решений.

Структура данных генетического алгоритма состоит из набора хромосом. Хромосома, как правило, представляет собой битовую строку, так что термин строка часто заменяет понятие «хромосома». Вообще говоря, хромосомы генетических алгоритмов не ограничены только бинарным представлением. Известны другие реализации, построенные на векторах вещественных чисел. Несмотря на то, что для многих реальных задач, видимо, больше подходят строки переменной длины, в настоящее время структуры фиксированной длины наиболее распространены и изучены.

Для иллюстрации идеи мы ограничимся только структурами, которые являются битовыми строками. Каждая хромосома (строка) представляет собой последовательное объединение ряда подкомпонентов, которые называются генами. Гены расположены в различных позициях или локусах хромосомы, и принимают значения, называемые аллелями - это биологическая терминология. В представлении хромосомы бинарной строкой, ген является битом этой строки, локус - есть позиция бита в строке, а аллель - это значение гена, 0 или 1. Биологический термин «генотип» относится к полной генетической модели особи и соответствует структуре в генетическом алгоритме. Термин «фенотип» относится к внешним наблюдаемым признакам и соответствует вектору в пространстве параметров задачи. В генетике под мутацией понимается преобразование хромосомы, случайно изменяющее один или несколько генов. Наиболее распространенный вид мутаций - случайное изменение только одного из генов хромосомы. Термин кроссинговер обозначает порождение из двух хромосом двух новых путем обмена генами. В литературе по генетическим алгоритмам также употребляется термин кроссовер, скрещивание или рекомбинация. В простейшем случае кроссинговер в генетическом алгоритме реализуется так же, как и в биологии. При скрещивании хромосомы разрезаются в случайной точке и обмениваются частями между собой. Например, если хромосомы $(11,12,13,14)$ и (0, 0, 0, 0) разрезать между вторым и третьим генами и обменять их части, то получатся следующие потомки (11, 12, 0, 0) и (0, 0, 13, 14).

Основные структуры и фазы генетического алгоритма. Приведем простой иллюстративный пример – задачу максимизации некоторой функции двух переменных $f(x_1, x_2)$, при ограничениях: $0 < x_1 < 1$ и $0 < x_2 < 1$. Обычно методика кодирования реальных переменных x_1 и x_2 состоит в преобразовании их в двоичные целочисленные строки определенной длины, достаточной для того, чтобы обеспечить желаемую точность. Предположим, что 10-ти разрядное кодирование достаточно для x_1 и x_2. Установить соответствие между генотипом и фенотипом можно, разделив соответствующее двоичное целое число на $2^{10} - 11$. Например,

0000000000 соответствует $0/1023$ или 0 , тогда как 1111111111 соответствует $1023/1023$ или 1.

Оптимизируемая структура данных есть 20-ти битовая строка, представляющая собой конкатенацию (объединение) кодировок x_1 и x_2. Пусть переменная x_1 размещается в крайних левых 10 -ти битах строки, тогда как x_2 размещается в правой части генотипа особи. Таким образом, генотип представляет собой точку в 20-ти мерном целочисленном пространстве (вершину единичного гиперкуба), которое исследуется генетическим алгоритмом. Для этой задачи фенотип будет представлять собой точку в двумерном пространстве параметров (x_1, x_2).

Чтобы решить задачу оптимизации нужно задать некоторую меру качества для каждой структуры в пространстве поиска. Для этой цели используется функция приспособленности. При максимизации целевая функция часто сама выступает в качестве функции приспособленности, для задач минимизации целевая функция инвертируется и смещается в область положительных значений.

Рассмотрим фазы работы простого генетического алгоритма. В начале случайным образом генерируется начальная популяция (набор хромосом). Работа алгоритма представляет собой итерационный процесс, который продолжается до тех пор, пока не будет смоделировано заданное число поколений или выполнен некоторый критерий останова. В каждом поколении реализуется пропорциональный отбор приспособленности, одноточечная рекомбинация и вероятностная мутация. Пропорциональный отбор реализуется путем назначения каждой особи (хромосоме) i вероятности $P(i)$, равной отношению ее приспособленности к суммарной приспособленности популяции (по целевой функции):

$$P(i) = \frac{f(i)}{\sum_{i=1}^{n} f(i)}.$$

Затем происходит отбор (с замещением) всех n особей для дальнейшей генетической обработки, согласно убыванию величины $P(i)$. Простейший пропорциональный отбор реализуется с помощью рулетки (Гольдберг, 1989 г.). «Колесо» рулетки содержит по одному сектору для каждого члена популяции, а размер i-ого сектора пропорционален соответствующей величине $P(i)$. При таком отборе члены популяции, обладающие более высокой приспособленностью, будут выбираться чаще по вероятности.

После отбора выбранные n особей подвергаются рекомбинации с заданной вероятностью P_c, при этом п хромосом случайным образом разбиваются на $n/2$ пар. Для каждой пары с вероятность P_c может быть выполнена рекомбинация. Если рекомбинация происходит, то полученные потомки заменяют собой родителей. Одноточечная рекомбинация работает следующим образом. Случайным образом выбирается одна из точек разрыва, т. е. участок между соседними битами в строке. Обе родительские структуры разрываются на два сегмента по этому участку. Затем

соответствующие сегменты различных родителей склеиваются и получаются два генотипа потомков.

После стадии рекомбинации выполняется фаза мутации. В каждой строке, которая подвергается мутации, каждый бит инвертируется с вероятностью P_m. Популяция, полученная после мутации, записывается поверх старой, и на этом завершается цикл одного поколения в генетическом алгоритме.

Полученное новое поколение обладает (по вероятности) более высокой приспособленностью, наследованной от «хороших» представителей предыдущего поколения. Таким образом, из поколения в поколение, хорошие характеристики распространяются по всей популяции. Скрещивание наиболее приспособленных особей приводит к тому, что исследуются наиболее перспективные участки пространства поиска. В результате популяция будет сходиться к локально оптимальному решению задачи, а иногда, может быть, благодаря мутации, и к глобальному оптимуму.

Теперь мы можем сформулировать основные шаги генетического алгоритма.

Генетический алгоритм:

1. Создать начальную популяцию.

2. Цикл по поколениям пока не выполнено условие останова //цикл жизни одного поколения.

3. Оценить приспособленность каждой особи.

4. Выполнить отбор по приспособленности.

5. Случайным образом разбить популяцию на две группы пар.

6. Выполнить фазу вероятностной рекомбинации для пар популяции и заменить родителей.

7. Выполнить фазу вероятностной мутации.

8. Оценить приспособленность новой популяции и вычислить условие останова.

9. Объявить потомков новым поколением.

10. Конец цикла по поколениям.

III. Муравьиные алгоритмы

Муравьиные алгоритмы представляют собой новый перспективный метод решения задач оптимизации, в основе которого лежит моделирование поведения колонии муравьев. Колония представляет собой систему с очень простыми правилами автономного поведения особей. Однако, несмотря на примитивность поведения каждого отдельного муравья, поведение всей колонии оказывается достаточно разумным. Эти принципы проверены временем - удачная адаптация к окружающему миру на протяжении миллионов лет означает, что природа выработала очень удачный механизм поведения. Исследования в этой области начались в середине 90-х годов XX века, автором идеи является Марко Дориго из Университета Брюсселя, Бельгия.

Муравьи относятся к социальным насекомым, образующим коллективы. В биологии коллектив муравьев называется колонией. Число муравьев в колонии может достигать нескольких миллионов, на сегодня известны суперколонии муравьев (Formica lugubrus), протянувшиеся на сотни километров. Одним из подтверждений оптимальности поведения колоний является тот факт, что сеть гнезд суперколоний близка к минимальному остовному дереву графа их муравейников.

Основу поведения муравьиной колонии составляет самоорганизация, обеспечивающая достижения общих целей колонии на основе низкоуровневого взаимодействия. Колония не имеет централизованного управления, и ее особенностями является обмен локальной информацией только между отдельными особями (прямой обмен пища, визуальные и химические контакты) и наличие непрямого обмена, который и используется в муравьиных алгоритмах.

Непрямой обмен – стигмержи (stigmergy), представляет собой разнесенное во времени взаимодействие, при котором одна особь изменяет некоторую область окружающей среды, а другие используют эту информацию позже, в момент, когда они в нее попадают. Биологи установили, что такое отложенное взаимодействие происходит через специальное химическое вещество – феромон (pheromone), секрет специальных желез, откладываемый при перемещении муравья. Концентрация феромона на тропе определяет предпочтительность движения по ней. Адаптивность поведения реализуется испарением феромона, который в природе воспринимается муравьями в течение нескольких суток. Мы можем провести некоторую аналогию между распределением феромона в окружающем колонию пространстве, и «глобальной» памятью муравейника, носящей динамический характер.

Идея муравьиного алгоритма – моделирование поведения муравьев, связанное с их способностью быстро находить кратчайший путь от муравейника к источнику пищи и адаптироваться к изменяющимся условиям, находя новый кратчайший путь. При своем движении муравей метит свой путь феромоном, и эта информация используется другими муравьями для выбора пути. Это элементарное правило поведения и определяет способность муравьев находить новый путь, если старый оказывается недоступным. Дойдя до преграды, муравьи с равной вероятностью будут обходить ее справа и слева. То же самое будет происходить и на обратной стороне преграды. Однако, те муравьи, которые случайно выберут кратчайший путь, будут быстрее его проходить, и за несколько передвижений он будет более обогащен феромоном. Поскольку движение муравьев определяется концентрацией феромона, то следующие будут предпочитать именно этот путь, продолжая обогащать его феромоном, до тех пор, пока этот путь по какой-либо причине не станет доступен. Очевидная положительная обратная связь быстро приведет к тому, что кратчайший путь станет единственным маршрутом движения большинства муравьев. Моделирование испарения феромона – отрицательной обратной связи, гарантирует нам, что найденное локально оптимальное решение не будет единственным - муравьи будут искать и другие пути. Если мы моделируем процесс такого поведения на

некотором графе, ребра которого представляют собой возможные пути перемещения муравьев, в течение определенного времени, то наиболее обогащенный феромоном путь по ребрам этого графа и будет являться решением задачи, полученным с помощью муравьиного алгоритма.

1.2. Языки программирования

Систематическое изучение теории и практики современных языков программирования (ЯП): принципов их построения, оценки и использования [27], целесообразно по следующим причинам [27]:

1. Понимание общих идей построения и применения языка – это ключ к легкому и глубокому освоению конкретного языка как основного профессионального инструмента программиста.

2. Понимание принципов построения языков – необходимое условие предоставления пользователю максимума услуг при минимуме затрат.

3. Изучение ЯП дает знание и понимание разнообразных подходов к программированию. Каждый ЯП – это своя философия. свой взгляд на деятельность программиста, отражение определенной технологии программирования.

4. Понятия и тенденции в области ЯП точно отражают особенности программирования как науки, искусства и ремесла. Фактически мы обсуждаем основные принципы и понятия программирования с языковой точки зрения.

5. Эти знания позволят оценить и выбрать ЯП для выполнения конкретного проекта, учесть затраты на его освоение и распространение написанных программ.

Algol (международная группа ученых, руководитель Петер Наур) – во главу угла был поставлен академический подход. Творцы языка акцентировали внимание на строгости определения и изяществе языковых конструкций. Самое важное, считали они, – это ясно и точно сформулировать алгоритм. В стороне остались технические вопросы: ввод-вывод, компоновка программ из модулей.

Fortran (фирма IBM, руководитель Джон Бэкус) – яркий пример прагматического подхода. Строгости и изяществу языка здесь уделялось мало внимания. Основной акцент был сделан на модульности, организации ввода-вывода.

Basic (Дартмутский колледж. Джон Кемени и Томас Курц) – реализовал диалоговый подход к взаимодействию с компьютером. Главными целями были объявлены простота языка, дружественность к пользователю. Мало того, язык поддерживал совсем иной принцип взаимодействия пользователя с компьютером. Предлагалось обеспечить каждого пользователя (неслыханное дело!) своим терминалом для взаимодействия с машиной.

Существует огромное количество языков программирования, среди которых выделяют дюжину самых популярных. Каждый язык демонстрирует свою специфику, имеет свои достоинства и недостатки. Очень легко потеряться в море информации о реальных языках программирования. Чтобы этого не случилось, будем изучать в основном модели языков, активно использовать моделирование как аппарат абстракции-конкретизации.

Создавая понятие (в интересах какой-либо задачи), мы отвлекаемся (абстрагируемся) от несущественных свойств реальных объектов, фиксируя только существенные свойства.

Рассмотрим модель часов. Выделим только одно свойство – «показывать время». В этой модели мы, к примеру, отказываемся от остальных свойств реальных часов: формы, цвета, материала, изготовителя.

Приводя пример, мы. напротив, конкретизируем абстрактное понятие, снабжая его второстепенными, но важными в данной ситуации деталями:

1. Конкретное выполнение процедуры происходит при конкретных значениях ее параметров.

2. Конкретный пример языка программирования – C++ – включает в себя конкретные конструкции.

Абстрактное понятие (модель) проще конкретного объекта, оно отражает лишь некоторую точку зрения на объект.

При решении другой задачи точка зрения на объект может быть другой, может потребоваться другая модель того же объекта.

Модель часов в качестве украшения включает свойства: форму, цвет, фирму, размер. Отказываемся от свойства «показывать время».

Модель размещения процедуры в памяти. Отказываемся от смысла выполняемых в ней действий.

Непроизвольный переход от одной модели к другой может быть источником недоразумений!

Вы говорите, что часы «плохие», потому что некрасивые, а я говорю «хорошие», так как они отлично работают. Если мы решаем «часы хорошие или плохие?», нам надо задаться критерием, уместным в рамках конкретной модели.

Итак, аппарат абстракции-конкретизации – это инструмент, которым надо пользоваться умело и осторожно. Мы будем подробно обсуждать источники недоразумений, возникающих при применении абстракции-конкретизации для изучения языков программирования. Однако предварительно сформулируем панораму определений предмета изучения – языка программирования.

Язык программирования (ЯП) – это знаковая система для планирования поведения компьютера.

Знаковая система – это совокупность соглашений, определяющих набор знаковых ситуаций. Знаковая ситуация первичное понятие семиотики, ее элементами являются знак и денотат.

Знак – это обозначение или имя, *денотат* – значение, смысл.

Примеры знаков и денотатов: сообщение – смысл; буква – звук; слово – понятие; sqrt – вычисление квадратного корня; if-then-else – разветвление вычислений.

Правила, определяющие структуру допустимых знаков, называют *синтаксисом* языка.

Правила, определяющие денотаты, которые соответствуют допустимым знакам, называют *семантикой* языка.

Опираясь на введенные понятия, можно предложить модифицированную формулировку языка программирования.

Язык программирования – это правила образования знаков (синтаксис) и согласованные с ними правила образования денотатов (семантика).

В ЯП знаки – это элементы программ, а денотаты – характеристики поведения исполнителя (компьютера).

Язык программирования (практическое определение) – это средство общения между человеком (пользователем) и компьютером (исполнителем).

С помощью ЯП формируются сообщения для компьютера. Эти сообщения должны быть понятны компьютеру. Возможные ошибки разделяют на синтаксические, семантические и прагматические .

Синтаксические ошибки возникают, если автор сообщения подразумевает одну структуру сообщения, а получатель – другую. Устраняются легко. В ЯП таких ошибок не бывает, так как их создатели используют принцип синтаксической однозначности (правильный текст должен иметь единственную допустимую структуру).

Семантические ошибки связаны с искажением смысла элементов сообщения, а прагматические ошибки – с искажением цели (назначения) элемента или сообщения.

Прагматические ошибки – самые трудные. Их примеры: часы хорошие-плохие, выяснение смысла программы без знания се назначения.

Для исключения ошибок необходимо, чтобы отправитель и адресат пользовались:

- одинаковыми правилами разложения сообщения на составляющие (предмет синтаксиса);

- согласованными правилами, придающими сообщению смысл (предмет семантики);

- согласованными целевыми установками (предмет прагматики).

Язык программирования (технологическое определение) – инструмент для производства программных услуг.

Программирование нацелено на обслуживание пользователя. Принцип хорошего обслуживания: «Клиент всегда прав». Это означает, что программы должны быть дружественными к пользователю.

Программы должны быть:

- надежными (содержать мало ошибок);

- устойчивыми (сохранять работоспособность в неблагоприятных условиях эксплуатации, ограниченности ресурсов, перегрузок. ошибок пользователя, сбоев и отказов аппаратуры);

- заботливыми (должны уметь объяснять свои действия и ошибки пользователя).

Известно, что создание программ – дорогое занятие. С современной точки зрения, аппаратура компьютера по относительной стоимости сравнима с упаковкой, в которую «заворачивается» программное изделие. Почему? Причина – сложность программирования. Известны два источника сложности.

Семантический разрыв – разрыв между уровнем элементарных операций компьютера и уровнем потенциально возможных услуг. Компьютер – очень быстрый исполнитель. Отсюда соблазн программировать его на оказание сложных услуг (с помощью очень сложных программ). В то же время возможности человека при работе со связанными объектами ограничены числом $\text{ИНГВЕ} = 7 \pm 2$. Выход из положения – создавать для определенного класса услуг подходящий ЯП (со сложными операциями) и программировать на таком ЯП.

Незнание компьютером реального мира. Поэтому компьютер не может контролировать соответствие указаний программиста поставленным задачам (целям), ради достижения которых они сформированы.

Примеры серьезных программных ошибок:

1. Космический аппарат «Маринер-1» (22 июля 1962 г.). Цель – Венера. Уничтожен через 294.5 секунды после старта.

2. Космический аппарат «Маринср-2» (27 августа 1962 г). Источник ошибки – пробел и пропуск запятой в Фортран-программе управления (оператор цикла DO).

3. Ракета-носитель «Ариан-5» (4 июня 1996 г.). Взорвана при старте. Стоимость ошибки - $500 000 000.

4. Конвейерные отключения электричества на северо-востоке Северной Америки (14 августа 2003 г.). Стоимость – 10 миллиардов долларов.

Средство борьбы с семантическим разрывом аппарат абстракции-конкретизации языка. Этот аппарат – основа ориентации на проблему. Например, в языке Fortran средство абстракции – подпрограмма, а соответствующее средство конкретизации – обращение к ней с фактическими параметрами.

Средство борьбы с незнанием реального мира аппарат прогноза-контроля языка. Этот аппарат – основа повышения надежности и устойчивости программ. Подобный аппарат в языке ассемблера очень слаб. В языке Fortran средством прогнозирования являются встроенные типы данных, соответствующий контроль предусмотрен

семантикой языка, но средств управления таким контролем нет. В языках искусственного интеллекта такой аппарат прямо предназначен для предоставления знаний о мире, целях деятельности и контроля действий (как программы, таки пользователя).

Теперь мы можем сформулировать технологический критерий качества ЯП: язык тем лучше, чем проще производство (на его основе) программных услуг.

Современное человеческое общество все больше и больше зависит от компьютерных систем, управляемых программами. Эти системы весьма настойчиво проникают во все сферы человеческой жизни. Они буквально окружают человека и на работе, и дома, и на отдыхе.

Известно, что аппаратура современных компьютерных систем очень сложна, и вместе с тем считают: сложность программного обеспечения превосходит аппаратную сложность более чем на порядок. Ф. Брукс, известнейший авторитет в данной области, утверждает: «Сложность программного обеспечения является существенным, а не второстепенным свойством».

С одной стороны, программы абстрактны и нематериальны. Они не имеют физической природы и присущих этой природе ограничений, кажутся человеку-творцу очень податливой «глиной», из которой можно «вылепить» все, что угодно. С другой стороны, сложность программного обеспечения может превосходить возможности человеческого разума.

Основной инструмент для создания программ – язык программирования. Выбор подходящего языка программирования для решения задачи определяется множеством причин, но главной причиной является принадлежность задачи к конкретной предметной области. Обсудим наиболее представительные области, в которых применяются языки программирования.

Первые компьютеры, появившиеся в середине XX века, изначально создавались для научных целей. Задачи из этой области сводились к решению различных математических уравнений. К ним относились задачи численного анализа, решения дифференциальных и интегральных уравнений и задачи статистики. Обычно научные программы обрабатывают простые структуры данных и производят большое количество арифметических вычислений, выполняемых над числами с плавающей точкой. 11аиболсе часто употребляются такие структуры, как массивы и матрицы; из управляющих структур чаще других используются циклы со счетчиком и условные операторы. Языки «научного» программирования разрабатывались именно для удовлетворения таких потребностей. Здесь всегда доминировал Fortran, поскольку его синтаксис был очень близок к математическому языку ученых. Язык Algol 60 и большинство его потомков также предназначались для подобных целей, хотя могли применяться и в других родственных областях. Однако для научных вычислений особо важна эффективность, а ни один из созданных языков не оказался лучше языка Fortran. В XXI веке Fortran по-прежнему не сдает своих позиций, однако языки Java и C++ достаточно успешно конкурируют с его современной версией – Fortran 2008.

Обработка деловой информации

Задачи обработки деловой информации, называемые бизнес-задачами, ориентированы на учет поступления заказов, управление ресурсами и персоналом, деловое планирование, анализ риска и оценку возможных вариантов, а также начисление зарплаты. Бизнес-программы обеспечивали считывание больших объемов данных, собранных за длительный период времени, и сохраняли их па магнитных лентах и дисках, а также формировали новые данные, обновляемые в результате небольших преобразований. Языки бизнес-программирования нацеливались на генерацию отчетов со сложной структурой, точные способы описания и храпения десятичных чисел и символьных данных, а также на арифметические действия с десятичными числами. Для этих задач был разработан язык Cobol, который и сейчас широко используется. Разработчики этого языка приложили немало усилий для реализации корректной обработки данных.

В паши дни основным языком бизнес-программирования по-прежнему считается Cobol, хотя иногда применяют и другие языки (например, C, C++). Однако позицию языков при обработке деловой информации сильно потеснили электронные таблицы, придуманные для персональных компьютеров. Если раньше на создание обычной программы делового планирования уходило несколько месяцев, то теперь аналитик может за несколько часов сформировать много электронных таблиц, решающих аналогичные задачи.

Дополнительным инструментом деловой обработки стали языки четвертого поколения 4GL (Fourth Generation Languages). Языки 4GL – это непроцедурные языки, специализированные под конкретные задачи обработки деловой информации и рассматривающие программу как спецификацию требуемого результата, а не как спецификацию действий, приводящих к достижению этого результата. Поддерживающая язык 4GL среда переводит спецификацию результата в машинно-выполнимую программу. Языки 4GL обеспечивают быстрое создание оконного интерфейса и простой доступ к содержимому базы данных, генерацию графических входных форм-бланков и выходных отчетов.

Искусственный интеллект (ИИ) – это предметная область, главная цель которой состояла в моделировании мыслительной деятельности человека. Отличительной особенностью возникавших здесь задач являлись символьные, а не числовые вычисления. Под символьными вычислениями подразумевается обработка не чисел, а символов. Оказалось, что символьные вычисления удобнее производить с помощью связных списков данных, а не массивов. Достаточно часто символьные вычисления требуют большей гибкости, чем другие предметные области. Например. в некоторых задачах целесообразно создание целых фрагментов программы в период се выполнения.

Основным инструментом производимых вычислений здесь считается функция, а используемые языки программирования называются функциональными языками. Родоначальником языков функционального программирования стал язык LISP,

придуманный ученым Д. Маккарти (1965). Последователями языка LISP являются языки Scheme, Haskell ит. д. В 70-х годах для решения задач ИИ начали применять и альтернативный подход на основе языка логического программирования Prolog.

Все задачи системной области связаны с созданием и развитием операционных систем для компьютеров системным программным обеспечением (ПО). Понятно, что системное ПО широко используется, имеет доступ ко всем функциональным возможностям и ресурсам аппаратуры и должно эффективно функционировать. Как следствие, применяемые языки программирования должны обеспечивать быстрое выполнение программ и взаимодействие с аппаратными средствами.

Вначале для достижения максимальной эффективности системное программирование велось на языке ассемблера.

В настоящее время повсеместно применяется язык С и его более новый вариант С++. Например, операционная система UNIX полностью написана на языке С. Язык С обеспечивает очень эффективное выполнение программ и позволяет программисту получить полный доступ как к операционной системе, так и к аппаратуре. Он не обременяет программиста большим количеством ограничений. Профессионалы, занимающиеся системным программированием, обычно думают, что подобные ограничения им не нужны. Тем не менее некоторые специалисты считают язык С слишком опасным для программирования больших и важных систем.

Еще один язык – Ada – тоже проектировался для применения в системной области, однако из-за ряда субъективных и объективных причин не получил здесь статуса основного языка. Он востребован в системах аэрокосмического и военного назначения.

Близкой предметной областью считается управление технологическими процессами и оборудованием. С появлением дешевых микропроцессоров, используемых в автомобилях, светофорах, стиральных машинах, видеоиграх и электронных часах, возросла необходимость в языках, позволяющих создавать программы для работы в реальном времени. К таким языкам относятся С, Ada и С++.

Появление Всемирной паутины (WWW – World Wide Web) оказало сильное воздействие на географию компьютерных взаимодействий. По всему миру создаются серверы информационных архивов. Для получения информации пользователи подключаются к этим серверам через Интернет, а для ее обработки используют локальные клиентские машины. Чтобы пользователь мог загрузить программу, а поставщик программного обеспечения имел возможность получить плату за ее использование, стал необходим язык программирования, позволяющий вести диалог между клиентским компьютером и сервером. Развитие электронной коммерции находится в полной зависимости от языков с такими возможностями.

Изначально веб-страницы были статическими документами: можно было лишь просмотреть текст или рисунки. Создавались страницы с помощью языка разметки HTML, который не является языком программирования. Для доступа к другой странице пользователь мог щелкнуть на ее адресе URL (Uniform Resource Locator).

Однако для развития электронной коммерции информация должна передаваться в обоих направлениях между клиентской машиной и сервером, поэтому веб-страницы должны были стать активными, динамическими. Подобные возможности обеспечиваются такими языками программирования, как Perl, PHP и Java.

Использование WWW снова поставило перед языками проблему производительности:

- линии связи пользователя с Интернетом, как правило, ограничивают скорость передачи данных;

- из-за подключения большого количества пользователей сервер может оказаться перегружен.

Для исключения влияния таких ситуаций можно обрабатывать информацию на клиентской машине, а не на сервере. Чтобы разгрузить сервер за счет клиентской машины, он должен переслать клиенту небольшую исполняемую программу. Но ведь серверу неизвестно, каким компьютером является клиентская машина, поэтому не ясно, какого вида должна быть исполняемая программа. Традиционно используется то решение, которое предлагает в этой ситуации язык Java, созданный для работы в среде WWW. Кроме того, заметим, что исполняемые программы (аплеты) языка Java были специально разработаны для обеспечения конфиденциальности частной жизни пользователя. Аплету, например, не разрешается выполнять запись или удаление файлов на клиентской машине.

1.3. Базовые алгоритмические конструкции

Удивительно, но основными «рабочими лошадками» в императивных программах являются *выражения* и *операторы присваивания*: выражения определяют порядок вычисления значений, а операторы присваивания обеспечивают занесение значений в некоторые переменные [27].

Основным назначением присваивания считается присвоение значению объекта данных (то есть области памяти, выделенной для объекта) значения (то есть значения объекта данных) определенного выражения. Синтаксис представления явного присваивания от языка к языку существенно меняется: Pascal, Ada – $Y := X$; C, C++, Java, Visual Basic, FORTRAN, PL/1, Prolog, ML – $Y = X$; COBOL – $MOVE\ \ X\ \ \text{TO}\ \ Y$; APL – $Y \leftarrow X$; LISP – $(SETQ\ \ Y\ \ X)$.

В семействе языков, сходных но синтаксису с языком C, присваивание считается просто операцией, поэтому можно написать: $y = x = 5$. Поскольку присваивание является правоассоциативной операцией, данная запись означает $y = (x = 5)$, и ей соответствует следующий порядок вычислений:

1. Переменной x присваивается значение 5.
2. Выражение $(x = 5)$ возвращает значение 5.
3. Переменной y присваивается значение 5.

И все же чаще всего присваивание используется только в качестве отдельного оператора. Примером может служить оператор присваивания в языке Pascal: $y \coloneqq 2 + x; a = 4 + 5 + 3$.

Другие возможности здесь исключены, так как операция присваивания языка Pascal не возвращает никакого явного результата.

В большинстве языков присваивания применяются лишь отдельно. Однако, как показано в табл. 6 в языке С их комбинируют с другими арифметическими операциями.

Таблица 1. Комбинированные операции присваивания в языке С

Синтаксис	Описание
$Y = X$	Присваивает значение переменной X значению переменной Y, возвращает значение переменной Y
$Y+= X$	Увеличивает значение переменной Y на значение переменной X $(Y = Y + X)$, возвращает новое значение
$Y-= X$	Уменьшает значение переменной Y на значение переменной X $(Y = Y - X)$, возвращает новое значение
$Y *= X$	Умножает значение переменной Y на значение переменной X $(Y = Y \cdot X)$, возвращает новое значение
$Y/= X$	Делит значение переменной Y на значение переменной X $(Y = Y / X)$, возвращает новое значение

Такие комбинированные операции присваивания позволяют сократить запись присваиваемых выражений.

Очень интересные возможности дает применение в операторах присваивания для языка С операций изменения значения переменных на единицу. Речь идет об операции увеличения значения $(++)$ и операции уменьшения значения $(--)$. Первая называется операцией инкремента, вторая операцией декремента.

Возможна префиксная и постфиксная формы инкремента и декремента:

1. Если операции инкремента и декремента записаны в префиксной форме, то переменная сначала изменяется, а затем ее новое значение используется в выражении.

2. Если операции инкремента и декремента записаны в постфиксной форме, то в выражении используется текущее значение переменной, и только после этого се значение изменяется.

Использование этих форм иллюстрирует табл. 7.

Таблица 2. Префиксная и постфиксная формы инкремента и декремента

Синтаксис	Описание
++A	Увеличивает значение переменной A на 1 и возвращает новое значение (A = A + 1), возвращает r-значение A

–A	Уменьшает значение переменной A на 1 и возвращает новое значение (A = A - 1), возвращаетr-значение A
A++	Возвращает значение переменной A. затем увеличивает ее значение на 1 (сначала возвращает r-значение A, затем A = A + 1)
A–	Возвращает значение переменной A, затем уменьшает ее значение на 1 (сначала возвращает r-значение A, затем A = A - 1)

Если операции инкремента и декремента осуществляются в виде отдельного оператора, то обе формы приводят к одинаковому результату.

Большинство языков программирования содержат операторы для чтения вводимых пользователем данных с клавиатуры, из каналов связи или из файлов. Эти операторы также изменяют значение переменных через операции присваивания. Обычно синтаксис таких операторов выглядит как: read(file, data). В языке C вызов оператора scanf приводит к присваиванию переменной некоторого значения из стандартного файла ввода. В языке Perl операцию чтения вызывает упоминание стандартного входного файла. Например, $A = <STDIN> вызывает присваивание переменной $A очередной строки ввода.

Конечно, можно задать линейные вычисления, записав один за другим последовательность операторов присваивания. И все же количество полезных программ, состоящих исключительно из операторов присваивания, крайне ограничено. Чтобы сделать программы более гибкими и разнообразными, необходимы средства для организации разветвлений и повторного выполнения некоторых операторов. Операторы, предоставляющие такие возможности, обычно называют операторами управления.

Каким должен быть простой и универсальный набор операторов управления? Простой в том смысле, чтобы любой оператор можно было без труда освоить и легко применить при создании программы. Универсальность же набора должна покрывать все мыслимые и немыслимые программные приложения. Ответ па этот вопрос широко дискутировался на протяжении последних двадцати пяти лет XX века. По сути, речь шла о разумном расширении состава операторов языка программирования, эффективно увеличивающего легкость программирования за счет баланса простоты, универсальности и читабельности.

Как правило, оператор управления является структурой, в состав которой входит одна или несколько последовательностей операторов присваивания. Исследования 60-х годов XX века доказали, что структуры управления должны иметь один вход и один выход. Наличие нескольких входов в таких структурах делает программы менее читабельными и понятными.

Оператор перехода является простейшим средством изменения линейного порядка выполнения базовых операторов, он передает управление выполнением в любое указанное место программы.

Передача управления чаще всего указывается с помощью goto – оператора перехода на оператор с заданной меткой.

В языках программирования возможны два вида оператора goto:

1. Безусловный goto. Если в последовательности операторов встречается оператор безусловного перехода goto, например: goto NEXT_OP, то управление передается оператору, помеченному меткой NEXT_OP. а оператор, непосредственно следующий за оператором goto, не выполняется.

2. Условный goto. Если в последовательности операторов встречается оператор условного перехода goto, например: if X > Y then goto NEXT_OP, то управление передается оператору, помеченному меткой NEXT_OP, только в случае выполнения заданного условия X > Y.

Внешний вид разрешаемых меток меняется от языка к языку. В языке С для меток используют синтаксис идентификаторов, в языках FORTRAN и Pascal – беззнаковые целые константы.

Уместно заметить, за последние 40 лет оператору goto пришлось пережить очень трудные времена. Его использование при написании программ подвергалось суровой критике.

Конечно, оператор goto является самым мощным и простым оператором управления потоком вычислений. Однако, перефразируя знаменитое высказывание М. Жванецкого, одно неосторожное использование такого оператора и ты отец... проблемы. Он демонстрирует ошеломляющую мощь и гибкость. Достаточно сказать. что все другие структуры управления легко создаются с помощью оператора goto, но эта сила. увы. и определяет его предельную опасность. Применение goto без ясных ограничений может существенно понизить читабельность, надежность и простоту сопровождения программы.

Проблемы являются следствием способности оператора goto выполнять головокружительные «прыжки» через фрагменты программы. А ведь известно, что проще всего читается (и понимается) программа, операторы которой выполняются в порядке их размещения, то есть в порядке «сверху-вниз». Частичным решением проблемы может стать такое ограничение: оператору goto разрешают передавать управление в программе только вниз. Впрочем, в аварийных ситуациях ограничение может сниматься.

Впервые разъяснил опасность операторов goto Э. Дейкстра. который призывал чуть ли не сжечь его на костре и т. д. Среди сторонников более умеренных взглядов

назовем известного ученого Д. Кнута, предоставившего примеры, в которых эффективность оператора goto перевешивала его вред для читабельности.

Было создано несколько языков программирования без оператора goto (например. Java). Однако самые современные языки программирования включают операторы перехода. Был он включен и в язык С. несмотря па заявление его автора о «бесконечной неправильности» оператора перехода. В языках программирования без оператора goto предусматриваются дополнительные операторы управления, компенсирующие его отсутствие. Как правило, это операторы цикла или выхода из подпрограмм.

В языке С в качестве формы более безопасного управления переходами предусмотрен оператор break. Этот оператор может размещаться внутри операторов управления while, for. switch. Оператор break передаст управление оператору, непосредственно следующему за структурой управления, в которой он сам содержится. Считается, что использование оператора break не ухудшает читабельность программы.

Помимо этого, язык С имеет родственный оператору break оператор continue. Забегая вперед, отметим: данный оператор завершает выполнение текущей итерации цикла в операторах while и for и переводит управление в конец тела цикла. Таким образом, оператор break прерывает цикл, в то время как оператор continue досрочно передает управление на следующую итерации» цикла.

Иногда неправильное размещение оператора break внутри оператора управления switch нарушает стабильность вычисления программ на языке С. Снова забегая вперед, отметим, если нет оператора break, то от одной ветви оператора switch управление переходит к следующей. Может возникнуть путаница, из какой именно ветви (при наличии оператора break) следует выйти.

При выполнении программы происходит передача управления отдельным операторам. Каждый оператор в течение кванта времени управляет компьютером. Говорят, что вычисления задает *поток управления* в программе.

СПИСОК ЛИТЕРАТУРЫ

1. Игошин В.И. Математическая логика и теория алгоритмов : учеб, пособие для студ. высш. учеб, заведений / В. И. Игошин. – 2-е изд. стер. – М.: Издательский центр «Академия». 2008. – 448 с.

2. Глухов М. М., Шишков А. Б. Математическая логика. Дискретные функции. Теория алгоритмов: Учебное пособие. – СПб.: Издательство «Лань», 2012. – 416 с.

3. Геут К. Л. Математическая логика и теория алгоритмов : учсб.-метод. пособие / К. Л. Геут, С. С. Титов. – Екатеринбург : УрГУ ПС, 2017. – 85 с.

4. Мендельсон Э. Введение в математическую логику. – М.: Наука, 1976. – 320 с.

5. Зюзьков В. М. Математическая логика и теория алгоритмов : учебное пособие / В. М. Зюзьков. – Томск : Эль Контент, 2015. – 236 с.

6. Успенский В. Л. Труды но нематематике / В. А. Успенский. 2-е изд,, испр. и доп. : В 5 кн.– М. : ОГИ : Фоад «Магматические этюды», 2012. – Кн. 2 : Философия (с приложением трёх писем П. .П. Трауберг к автору). – 2014. – 566 с.

7. Лихтарников Л. М., Сукачева Т. Г. Математическая логика. Курс лекций. Задачник-практикум и решения: Учебное пособие. 4-е изд., стер. – СПб.: Издательство «Лань», 2009. – 288 с.

8. Вялый М., Подольский В., Рубцов А., Шварц Д., Шень А. Лекции по дискретной математике. 448 с. https://publications.hse.ru/mirror/pubs/share/direct/393719078.pdf

9. Пальчунов, Д. Е. Математическая логика и теория алгоритмов : учеб, пособие / Д. Е. Пальчунов ; Новосиб. гос. ун-т. - Новосибирск, 2016. – Ч. 1. – 96 с.

10. Барашев В.П.. Унучек С.А. Дискретная математика : Учебное пособие Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Московский государственный технический университет радиотехники, электроники и автоматики». – М., 2012. –268 с., электронное издание.

11. Чашкин А. В. Дискретная математика : учебник для учреждений высш. проф. образования / А. В. Чашкин. – М.: Издательский центр «Академия», 2012. – 352 с.

12. Гринченков Д.В. Математическая логика и теория алгоритмов для программистов : учебное пособие / Д.В. Гринченков, С.И. Потоцкий. – М.: КНОРУС, 2010. – 208 с.

13. Вайнштейн К. В. Математическая логика и теория алгоритмов : учеб, пособие / Ю. В. Вайнштейн, Т. Г. Пенькова, В. И. Вайнштейн. – Красноярск : Сиб. федер. ун-т, 2019. – 110 с.

14. Новиков Ф. А. Дискретная математика для программистов. Учебник для вузов. 3-е изд. – СПб.: Питер, 2009. – 384 с.

15. Марченков С. С. Основы теории булевых функций. М.: ФИЗМАТЛИТ, 2014. – 136 с.

16. Гаврилов Г. П., Сапоженко А. А. Задачи и упражнения по дискретной математике: Учеб, пособие. – 3-е изд., перераб. – М.: ФИЗМАТЛИТ, 2005. – 416 с.

17. Кормен Т. Х., Лензерсон, Ч. И., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ. 2-е издание : Пер. с англ. – М. : Издательский дом «Вильямс», 2011. – 1296 с.

18. Алексеев В.В. Теория алгоритмов. Учебно-методическое пособие. СарФТИ НИЯУ МИФИ, 2021. – 100 с.

19. https://education.yandex.ru/handbook/algorithms

20. Ахо А. В., Хопкрофт Д. Э., Ульман Д. Д. Структуры данных и алгоритмы : Пер. с англ. : М. : Издательский дом «Вильямс», 2003. – 384 с.

21. Граничин О.Н. Рандомизированные алгоритмы в задачах обработки данных и принятия решений. Системное программирование. Санкт-Петербургский государственный университет, Т. 6, № 1. С. 141–162.

22. Регеда В. В., Регеда О. Н. Основы алгоритмизации. Методические указания по курсу «Информатика». Пенза: Издательство Пензенского государственного университета, 2008. – 56 с.

23. Туаева М. Т. Алгоритмы. Основы алгоритмизации: Учебное пособие. – Владикавказ: Владикавказский торгово-экономический техникум, 2020. – 51 с.

24. Алгоритмы. Основные алгоритмические конструкции [Текст]: сб. задач / сост. С.А. Рогозин. – Челябинск: Изд-во Челяб. гос. пед. ун-та, 2008. 42 с.

25. Глухова Л.А. Основы алгоритмизации и программирования : лаб. практикум для студ. спец. I-40 01 01 «Программное обеспечение информационных технологий» дневн. формы обуч. В 4 ч. Ч. 3 / Л.А. Глухова, Е П. Фадеева, Е.Е. Фадеева. – Минск : БГУИР, 2007. – 52 с.

26. Макконнелл Дж. Основы современных алгоритмов. 2-е дополненное издание. Москва: Техносфера, 2004. – 368с.

27. Орлов С. А. Теория и практика языков программирования: Учебник для вузов. Стандарт 3-го поколения. – СПб.: Питер. 2013. – 688 с.

28. Анисимов А.Е. Ведение в алгоритмизацию задач / А.Е. Анисимов. – Ижевск: Изд-во «Удмуртский университет», 2017. – 44 с.

29. Конова Е. А. Алгоритмы и программы. Язык С++ : учебное пособие для СПО / Е. А. Конова, Г. А. Поллак. – 3-е изд., стер. – Санкт-Петербург : Лань, 2022. – 384 с.

30. Паклина В.М., Паклина Е.М. Основы алгоритмизации и программирования. – Екатеринбург: ГОУ ВПО УГТУ−УПИ, 2009. – 91 с.

31. Ворожцов А.В., Винокуров Н.А. Лекции Алгоритмы: Построение, анализ и реализация на языке программирования Си. – Москва: МФТИ, 2007. – 452 с.

32. Мусатов Д.В. Сложность вычислений : конспект лекций. – Москва: МФТИ, 2017. – 224 с.

33. Седжвик Р. Фундаментальные алгоритмы на С++. Анализ / Структуры данных / Сортировка / Поиск : Пер. с англ. / Роберт Седжвик. – К.: Издательство «ДиаСофт», 2001. – 688 с.

34. Имран А. 40 алгоритмов, которые должен знать каждый программист на Python. – СПб.: Питер. 2023. – 368 с.

35. Hetland M. L. Python Algorithms: Mastering Basic Algorithms in the Python Language. – 2010. – 337 p.

36. Пинчук В.П. Анализ и построение алгоритмов. Краткий конспект лекций для спец. КСС. – Запорожье: ЗНТУ, 2012.

37. Филиповский В.М. Основы программирования и алгоритмизации. Ч. 2. Практикум по алгоритмизации: Учебное пособие. – СПб.: СПбПУ. 2022. – 71 с.

38. Кормен Т. Х. Алгоритмы: вводный курс. : Пер. с англ. – М. : ООО «И.Д. Вильямс», 2014. – 208 с.

39. Стивенс Р. Алгоритмы. Теория и практическое применение / Род Стивене. – Москва : Издательство «Э». 2016. – 544 с.